

Can LLMs Implement Architectural Tactics? Early Results

Anonymous Author(s)

Abstract

Software maintainability is a critical quality attribute that directly affects system evolution cost and reliability. Improving maintainability often requires architectural-level modifications demanding specialized expertise and significant manual effort. While Large Language Models (LLMs) have shown strong capabilities in code understanding and generation, their potential for systematic architecture-level software quality improvement remains largely unexplored.

This paper presents an automated pipeline that leverages an LLM to improve software maintainability through architectural tactic implementation. The pipeline operates in three stages: (1) architecture detection, where the LLM analyzes repository structure and identifies the dominant architectural pattern; (2) architectural tactic selection, where the LLM chooses an appropriate tactic from a predefined catalog based on the detected architecture and expected maintainability impact; and (3) iterative incremental tactic implementation with automated verification. The pipeline follows a Pipes and Filters architecture and operates without manual intervention.

We applied the pipeline to «DATA 162 open-source Python backend repositories collected and filtered from GitHub. Of «DATA 121 repositories that completed the full pipeline, «DATA 22 showed measurable MI improvement (mean $\Delta MI = \text{«STAT} + 2.89$), «DATA 92 remained stable, and «DATA 7 showed minor degradation. A Wilcoxon signed-rank test confirmed a statistically significant positive shift ($\text{«STAT } W = 75.0, p = 0.001$), though with «STAT negligible effect size. Decomposability was the most frequently selected tactic («DATA 63.6%). Baseline maintainability was measured using the Radon static analysis tool, with the Maintainability Index as the primary metric.

This work contributes a reproducible methodology for LLM-driven architectural improvement and provides early empirical evidence on both the potential and limitations of LLMs for architecture-level modifications.

CCS Concepts

• **Software and its engineering** → **Software maintenance tools**; *Software architectures*; • **Computing methodologies** → *Natural language processing*.

ACM Reference Format:

Anonymous Author(s). 2026. Can LLMs Implement Architectural Tactics? Early Results. In *Proceedings of The 30th International Conference on Evaluation and Assessment in Software Engineering (EASE 2026)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
EASE 2026, Glasgow, United Kingdom

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

1 Introduction

Modern software engineering faces a persistent maintenance challenge, with studies indicating that 60–80% of total lifecycle costs are dedicated to post-deployment modifications [1]. While code-level refactoring is well-established, there is a significant gap in the automated application of higher-level design decisions—what we term the “Transformation Gap” between detecting the need for architectural change and implementing it automatically. This paper investigates the intersection of architectural tactics, quality attribute models, and the emerging capabilities of Large Language Models (LLMs) in automating structural transformations.

We address the following research questions:

- **RQ1:** Can an automated LLM-based pipeline detect architectural patterns and select appropriate tactics in real-world repositories?
- **RQ2:** To what extent do LLM-implemented architectural tactics improve software maintainability as measured by the Maintainability Index?
- **RQ3:** How do architecture type and tactic choice influence improvement outcomes?

The remainder of this paper is organized as follows: Section 2 reviews background and related work; Section 3 describes the methodology; Section 4 details the implementation; Section 5 presents results and discussion; Section 6 discusses threats to validity; and Section 7 concludes.

2 Background and Related Work

2.1 Architectural Tactics and Quality Attributes

Software architecture is fundamentally defined by its elements, form, and rationale [8], or alternatively, as a collection of components and connectors in a specific configuration [2]. Within these frameworks, *architectural tactics* serve as fine-grained design decisions intended to achieve specific quality attribute responses [1].

The ISO/IEC 25010 standard decomposes maintainability into modularity, analysability, modifiability, reusability, and testability [1]. Research by Rahmati suggests that while architectural patterns (e.g., MVC, Layered) provide a structural foundation, their effectiveness is highly context-dependent; misapplication can lead to a «SOURCE 10% increase in maintenance effort [10]. Consequently, the selection and implementation of specific tactics, such as “Use an Intermediary” or “Split Module,” are critical for targeted quality improvement [1].

2.2 LLMs in Software Refactoring

Recent multi-agent frameworks such as MANTRA have demonstrated an «SOURCE 82.8% success rate in producing compilable, test-passing refactored code [12]. However, empirical analysis of AI coding agents reveals a significant scale limitation: approximately «SOURCE 96.3% of current agentic refactorings are confined to a single file [3].

Research into prompt engineering indicates that while LLMs can identify up to «SOURCE 86.7% of refactoring opportunities, their performance in complex tactic synthesis remains limited [5]. In security-related tactic implementation, models may achieve high syntactic correctness (95%) but fail significantly in semantic correctness (5%) due to an inability to coordinate inter-procedural changes across multiple classes [11].

2.3 Research Gaps

The synthesis of current literature identifies three key gaps:

- **Contextual Constraints:** Current LLM-based tools operate primarily at the method or class level, lacking system-wide awareness for architectural restructuring [3, 7].
- **Measurement Inconsistency:** There is a documented lack of agreement among static analysis tools (less than 0.4% in some cases), complicating quality validation [4].
- **Implementation Void:** While methods exist for detecting the need for architectural tactics, automated LLM-driven mechanisms for concrete multi-file changes are lacking [6, 11].

3 Methodology

3.1 Research Design

This study employs a quantitative pre-test/post-test experimental design. The independent variable is the application of LLM-based architectural tactics; the dependent variable is software maintainability. Each repository serves as its own control, allowing paired-sample comparison before and after the intervention.

3.2 Experimental Pipeline

The pipeline is structured into seven sequential phases: (1) Automated repository selection; (2) Baseline static analysis; (3) Architectural pattern detection; (4) Tactic selection based on detected deficiencies; (5) Iterative tactic implementation; (6) Post-intervention static analysis; (7) Comparative statistical analysis.

3.3 Dataset Preparation

Repositories were sourced from public Python-based backend projects on GitHub. A multi-stage filtering process excluded toy projects and focused on maintainable industrial-like codebases [10]. Inclusion criteria required:

- Presence of a valid requirements.txt for dependency analysis;
- A minimum threshold of Python source files to ensure architectural complexity;
- Exclusion of ML-heavy projects (e.g., Jupyter Notebooks);
- Assessment of project maturity through star counts and commit history.

After filtering, «DATA 162 repositories were retained for the experiment.

3.4 Maintainability Measurement

The primary metric is the Maintainability Index (MI), calculated via the Radon framework, aligned with ISO/IEC 25010 [1]. Recognizing

the documented inconsistency among static analysis tools [4], we supplement MI with:

- **Structural Complexity:** Average fan-out and package coupling;
- **Documentation Quality:** Docstring coverage and README presence;
- **Delta Measurement:** $\Delta MI = MI_{after} - MI_{before}$.

3.5 LLM-Driven Transformation

The transformation leverages an LLM in three layers:

Architecture Detection: The model analyzes directory structure and module relationships to infer the architectural style (e.g., Layered, Modular Monolith) [2, 8].

Tactic Selection: Based on identified structural deficiencies, the system selects a tactic from the catalog established by Bass et al. [1].

Tactic Implementation: The implementation follows an *iterative incremental approach*, where the LLM performs small, verifiable changes. While literature suggests that Test-Driven Development (TDD) significantly improves reliability of LLM refactoring [9, 12], a full TDD cycle is designated for **future work**.

4 Implementation

The framework is implemented as an automated Python-based orchestration engine using a Pipes and Filters architecture. Target repositories are shallow-cloned, processed sequentially, and purged after data extraction to ensure isolation and reproducibility.

The LLM component uses «DATA Qwen3-coder-next (via Ollama, temperature = «DATA 0.2) with structured prompt engineering enforcing JSON-formatted outputs for automated parsing of architectural decisions. By maintaining a global architectural context in prompts, the implementation enables coordination of changes across multiple modules—addressing the single-file limitation identified in recent studies [9]. All artifacts, including LLM reasoning logs and intermediate code states, are preserved for full traceability.

5 Results and Discussion

5.1 Overall Quantitative Impact

Table 1 summarizes the overall results across «DATA 162 repositories. Of these, «DATA 41 («DATA 25.3%) failed to complete the pipeline (null mi_{after}), leaving «DATA 121 with paired before/after measurements.

Table 1: Descriptive statistics for repositories with paired MI measurements (N = «DATA 121).

Metric	MI_{before}	MI_{after}
Mean	«STAT 65.11	«STAT 65.59
Median	«STAT 68.42	«STAT 68.67
Std. Dev.	«STAT 19.17	«STAT 19.22
Mean ΔMI	«STAT +0.48	
Median ΔMI	«STAT 0.00	

A Wilcoxon signed-rank test on the «DATA 29 non-zero deltas yields «STAT $W = 75.0, p = 0.001$, indicating a statistically significant

positive shift in MI. However, the Cliff’s delta effect size is «STAT 0.017 (negligible), reflecting that the magnitude of improvement is small in aggregate.

As illustrated in Figure 1, a majority of projects maintained baseline stability. In «DATA 56.8% of completed cases, the MI remained unchanged, suggesting the LLM’s internal refactoring did not always produce changes detectable by Radon’s MI formula.

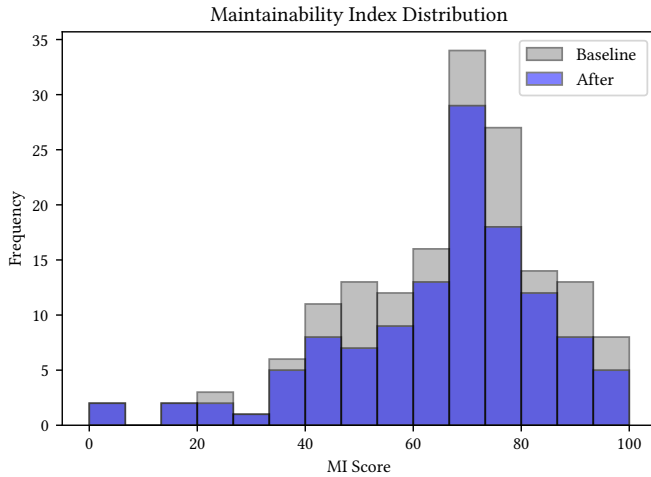


Figure 1: Distribution of MI scores: Baseline vs. Post-Intervention.

5.2 Success Cases and Tactic Efficacy

«DATA 22 repositories («DATA 13.6%) showed measurable MI improvement (Figure 2). The largest gain was in *get_subscribe* ($\Delta MI = \text{«DATA } +13.52$), where the **Decomposability** tactic decomposed large scripts into modular components—a change highly rewarded by the MI formula. Similarly, *whoogle-search* improved from «DATA 67.06 to «DATA 69.74 via **Localized Modification** in a layered architecture.

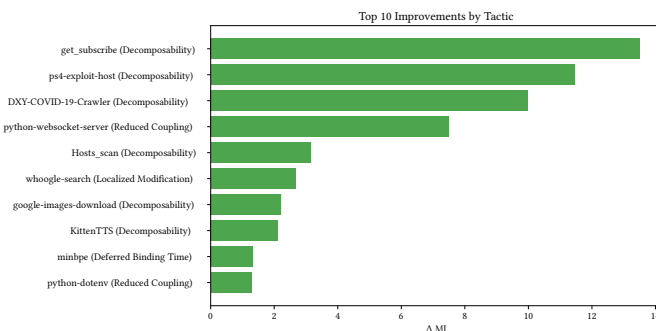


Figure 2: Top 10 MI improvements by applied tactic.

Table 2 breaks down results by tactic. **Reduced Coupling** showed the highest improvement rate («DATA 8/«DATA 18 improved,

«STAT 44.4%), while **Decomposability**—the most frequently selected tactic («DATA 103 repos)—had the highest null rate («DATA 29/«DATA 103, «DATA 28.2%), suggesting that decomposing unstructured code is a higher-risk transformation.

Table 2: Results by architectural tactic.

Tactic	N	Null	Impr.	Stable	ΔMI
Decomposability	«DATA 103	«DATA 29	«DATA 7	«DATA 63	«STAT +0.55
Localized Mod.	«DATA 32	«DATA 5	«DATA 6	«DATA 19	«STAT +0.13
Reduced Coupling	«DATA 23	«DATA 5	«DATA 8	«DATA 9	«STAT +0.70
Other	«DATA 4	«DATA 2	«DATA 1	«DATA 1	«STAT +0.67

5.3 Failure Rates and Robustness

Figure 3 shows that «DATA 25.3% of repositories resulted in pipeline failure (null status), representing cases where the LLM introduced syntax errors, broken dependencies, or changes that prevented execution. An additional «DATA 4.3% («DATA 7 repos) showed minor MI degradation (mean $\Delta MI = \text{«STAT } -0.78$). This underscores the necessity of verification mechanisms for automated architectural modifications.

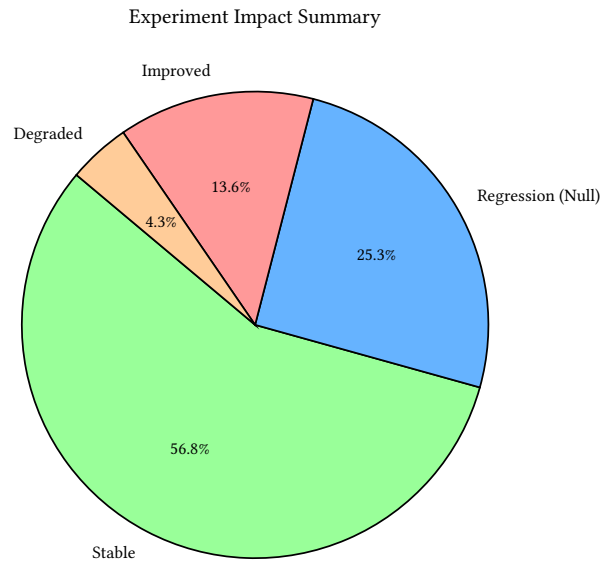


Figure 3: Intervention impact: improved, stable, degraded, and failed (null).

5.4 Architectural Influence

Figure 4 shows the dataset is dominated by **modular monolith** («DATA 51.2%) and **script-based** («DATA 43.8%) architectures. Script-based projects showed the highest mean improvement («STAT +0.81) but also the highest variance, with the top 3 gains all in

script-based repos «CLAIM (get_subscribe, ps4-exploit-host, DXY-COVID-19-Crawler). Modular monoliths showed more modest but consistent gains («STAT +0.22). «CLAIM All 7 degradation cases occurred in modular monolith repositories, suggesting that modifications to well-structured codebases carry higher risk of disrupting existing quality.

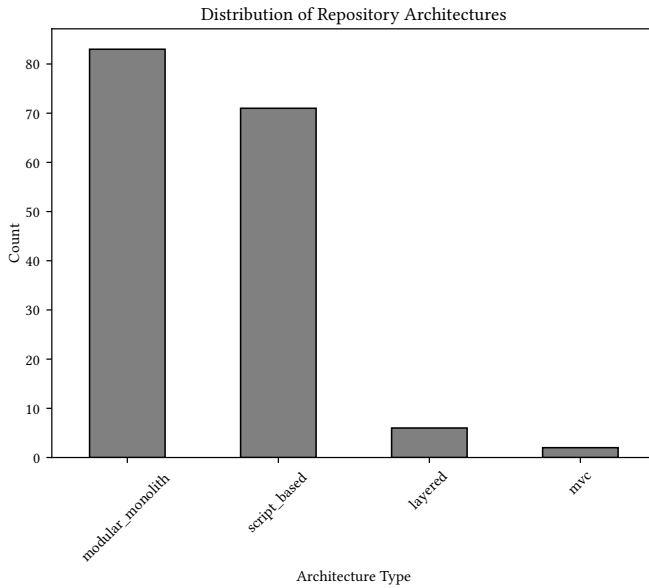


Figure 4: Distribution of baseline architectures.

6 Threats to Validity

Internal validity: Each repository uses itself as a control, but MI changes could be influenced by factors beyond the tactic applied (e.g., code formatting changes). A single LLM model was used without comparison to alternatives or manual implementation.

External validity: Results are limited to Python repositories from GitHub. The Qwen3-coder-next model’s behavior may not generalize to other LLMs, languages, or project types.

Construct validity: MI is a composite metric that may not capture all dimensions of maintainability. Architecture detection was performed by the LLM without human validation. The documented inconsistency among static analysis tools [4] further limits metric reliability.

Conclusion validity: While the Wilcoxon test shows statistical significance, the effect size is negligible. The effective sample of improved repositories («DATA 22 «DATA 162) is small, limiting the strength of conclusions.

7 Conclusion

This study investigated the potential of LLMs to automate architectural tactic implementation for software maintainability improvement. Regarding our research questions:

RQ1: The pipeline successfully detected architecture and selected tactics for all «DATA 162 repositories, demonstrating that

LLMs can perform architecture-level analysis on real-world codebases.

RQ2: LLM-implemented tactics produced a statistically significant but small positive effect on MI (mean $\Delta MI = \text{«STAT } +0.48$, «STAT $p = 0.001$). «DATA 22 repositories showed measurable improvement, though «DATA 56.8% remained unchanged and «DATA 25.3% failed.

RQ3: Script-based architectures showed the highest improvement potential, while modular monoliths exhibited both modest gains and all observed degradations. Reduced Coupling was the most effective tactic by improvement rate.

The high failure rate («DATA 25.3%) and prevalence of neutral results highlight limitations of current LLM pipelines for architecture-level modifications. Future work should integrate Test-Driven Development for verification, explore multi-LLM comparison, and extend beyond Python to evaluate generalizability.

References

- [1] Len Bass, Paul Clements, and Rick Kazman. 2021. *Software Architecture in Practice* (4th ed.). Addison-Wesley.
- [2] David Garlan and Mary Shaw. 1993. An introduction to software architecture. 1, 3 (1993).
- [3] Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. 2025. Agentic Refactoring: An Empirical Study of AI Coding Agents. In *arXiv preprint arXiv:2511.04824*. arXiv:2511.04824
- [4] Valentina Lenarduzzi, Fabiano Pecorelli, Nytyi Saarimaki, Savanna Lujan, and Fabio Palomba. 2023. A critical comparison on six static analysis tools: Detection, agreement, and precision. 198 (2023), 111575.
- [5] Bo Liu, Yanjie Jiang, Yuxia Zhang, Nan Niu, Guangjie Li, and Hui Liu. 2025. Exploring the potential of general purpose LLMs in automated software refactoring: An empirical study. *Automated Software Engineering* 32, 1 (2025). doi:10.1007/s10515-025-00500-0
- [6] Gastón Márquez, Hernán Astudillo, and Rick Kazman. 2022. Architectural tactics in software architecture: A systematic mapping study. *Journal of Systems and Software* 197 (2022), 111558. doi:10.1016/j.jss.2022.111558
- [7] Sofia Martinez, Luo Xu, Mariam Elnaggar, and Eman Abdullah AlOmar. 2025. Software refactoring research with large language models: A systematic literature review. *Journal of Systems and Software* (2025), 112762. doi:10.1016/j.jss.2025.112762
- [8] Dewayne E. Perry and Alexander L. Wolf. 1992. Foundations for the study of software architecture. 17, 4 (1992), 40–52. doi:10.1145/141874.141884
- [9] Yonnel Chen Kuang Piao, Jean Carlos Paul, L. Da Silva, Arghavan Moradi Dakhel, Mohammad Hamdaqa, and Foutse Khomh. 2025. Refactoring with LLMs: Bridging Human Expertise and Machine Understanding. In *arXiv preprint arXiv:2510.03914*. arXiv:2510.03914
- [10] Zahed Rahmati and Mohammad Tanhaei. 2021. Ensuring software maintainability at software architecture level using architectural patterns. 2, 1 (2021), 81–102. doi:10.22060/ajmc.2021.19232.1044
- [11] Seyyed Ehsan Shokri and Raffi Khatchadourian. 2024. IPSynth: Interprocedural Program Synthesis for Software Architecture Recovery and Architectural Tactics Detection. In *arXiv preprint arXiv:2403.10836*. arXiv:2403.10836
- [12] Yisen Xu, Feng Lin, Jinjiu Yang, Tse-Hsun Chen, and Nikolaos Tsantalis. 2025. MANTRA: Enhancing Automated Method-Level Refactoring with Contextual RAG and Multi-Agent LLM Collaboration. In *arXiv preprint arXiv:2503.14340*. arXiv:2503.14340